

01 — Connection Pooling Deep Dive

Table of Contents

1. [Learning Objectives](#)
 2. [Why Connection Pooling Is Needed](#)
 3. [PostgreSQL Process-Per-Connection Model](#)
 4. [PgBouncer Modes](#)
 5. [Pool Sizing Formula](#)
 6. [PgBouncer vs pgpool-II](#)
 7. [PgBouncer Configuration](#)
 8. [Monitoring PgBouncer](#)
 9. [max_connections in postgresql.conf](#)
 10. [Common Mistakes](#)
 11. [Best Practices](#)
 12. [Performance Considerations](#)
 13. [Interview Questions & Answers](#)
 14. [Exercises with Solutions](#)
 15. [Cross-References](#)
-

Learning Objectives

By the end of this section you will be able to:

- Explain why PostgreSQL's process-per-connection model makes connection management critical
 - Describe the three PgBouncer pooling modes and choose the right one for a workload
 - Apply the standard pool-sizing formula to a real server
 - Configure a production-ready `pgbouncer.ini`
 - Monitor pool health using built-in PgBouncer admin commands
 - Calculate safe `max_connections` values and understand the memory implications
-

Why Connection Pooling Is Needed

The Cost of a Raw Connection

Every time a client opens a TCP connection to PostgreSQL, the postmaster process **forks a new backend process**. On Linux this means:

```
Client connects → postmaster receives connection request
                  → fork() system call
                  → new OS process spawned (~5-10 MB shared memory + private heap)
                  → SSL handshake (if TLS)
                  → authentication (md5 / scram-sha-256)
                  → session setup (GUC loading, search_path, timezone)
                  → first query can run
```

Typical latency for this full cycle: **10-50 ms** on a busy server, plus the OS overhead of maintaining thousands of processes.

Memory Cost Per Connection

Component	Approx. Memory
Shared memory (from shared_buffers)	Negligible per connection
Backend private heap (work_mem, sort buffers)	5–10 MB baseline
Stack size	8 MB default on Linux
work_mem (worst case per sort node)	4 MB–1 GB (configurable)

With 500 connections and default settings:

$500 \times (8 \text{ MB stack} + 5 \text{ MB heap}) = 6.5 \text{ GB}$ just for idle backends

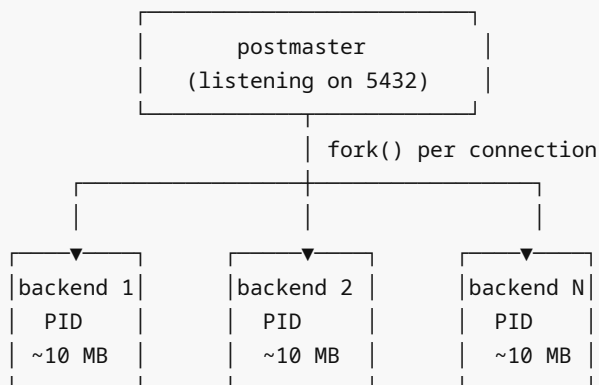
Connection Churn in Web Applications

A typical stateless REST API:

Request arrives → acquire DB connection → run 1-3 queries → release connection → respond

With 1000 req/s and 5ms average query time, only ~5 connections are active at any instant, yet naive connection-per-request code opens and closes **1000 connections per second**.

PostgreSQL Process-Per-Connection Model



Key characteristics:

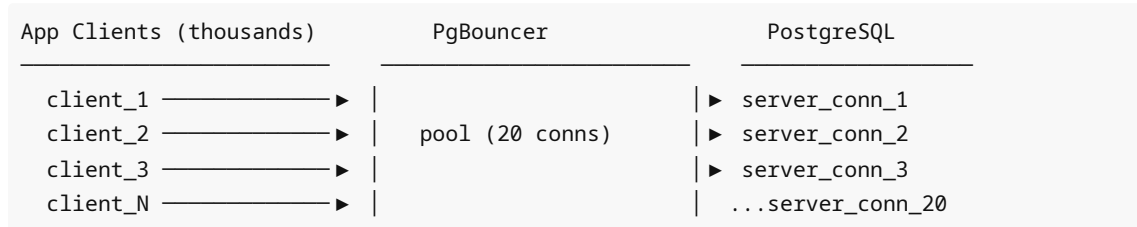
- Each backend has its **own process** (not a thread) — full isolation
- `fork()` on Linux uses copy-on-write, so shared memory is shared until written
- Context switches between OS processes are more expensive than user-space thread switches
- The postmaster itself is single-threaded for `accept()/fork()` logic

Why Threads Would Be Different

MySQL/MariaDB, SQL Server, and Oracle use a thread-per-connection model. Threads share virtual address space, making it cheaper to spawn but harder to isolate. PostgreSQL's process model provides superior crash isolation: a buggy backend cannot corrupt other backends' memory.

PgBouncer Modes

PgBouncer sits between the application and PostgreSQL, maintaining a **pool** of real server connections and multiplexing client connections onto them.



Session Pooling

Client connects → assigned a server connection for the ENTIRE session lifetime
Client disconnects → server connection returned to pool

Behaviour:

- One-to-one mapping while client is connected
- Allows: SET, LISTEN, prepared statements, advisory locks, temp tables
- No real multiplexing benefit during an active session
- Benefit: avoids the fork() cost on reconnect

Use when: You need full PostgreSQL session semantics (advisory locks, LISTEN/NOTIFY, session-level prepared statements).

Transaction Pooling

Client sends BEGIN → server connection assigned
Client sends COMMIT/ROLLBACK → server connection returned to pool
Between transactions → client holds NO server connection

Behaviour:

- A server connection is held only for the duration of a transaction
- Maximum multiplexing — 20 server connections can serve hundreds of clients
- **Restrictions:**
 - No session-level SET (use SET LOCAL inside transaction)
 - No LISTEN / NOTIFY via the pool
 - No session-level prepared statements (use protocol-level prepared statements OR disable them)
 - No advisory locks that span transactions
 - Temp tables are problematic (they persist on the server connection)

Use when: Stateless microservices, REST APIs, high-concurrency web apps.

Statement Pooling

Server connection returned to pool after EACH statement

Behaviour:

- Even `BEGIN / COMMIT` run on potentially different server connections
- Multi-statement transactions are **impossible** — each statement auto-commits
- Rarely used in practice

Use when: Only for read-only workloads with trivially simple queries, or legacy compatibility. Avoid in most applications.

Mode Comparison Matrix

Feature	Session	Transaction	Statement
Multi-statement transactions	Yes	Yes	No
SET (session-level)	Yes	No	No
SET LOCAL (transaction-level)	Yes	Yes	No
LISTEN/NOTIFY	Yes	No	No
Temp tables	Yes	Limited	No
Advisory locks (session)	Yes	No	No
Prepared statements (named)	Yes	No	No
Multiplexing efficiency	Low	High	Highest

Pool Sizing Formula

The canonical formula, from the HikariCP connection pool research and corroborated by PostgreSQL benchmarks:

```
pool_size = num_cores × 2 + effective_spindle_count
```

Where:

- `num_cores` = number of CPU cores available to PostgreSQL
- `effective_spindle_count` = number of HDDs (SSDs count as 1; NVMe ≈ 1-2)

Example Calculations

Server: 8-core CPU, NVMe SSD (spindle count ≈ 1)

```
pool_size = 8 × 2 + 1 = 17 ≈ 20 (round up for headroom)
```

Server: 32-core CPU, SAN with 10 spindles

```
pool_size = 32 × 2 + 10 = 74 ≈ 80
```

Why this formula works:

- Queries that are CPU-bound saturate all cores at `num_cores` concurrent connections
- I/O-bound queries park waiting for disk; the 2× multiplier accounts for this

- Additional connections beyond this number increase context switching without adding throughput
- Empirical benchmarks show diminishing returns and then regression beyond this threshold

PgBouncer Pool Math

max_connections (postgresql.conf) should be:
 $(\text{num_pgbouncer_pools} \times \text{pool_size}) + \text{reserved_superuser_connections}$

Example:
3 application pools × 20 conns each = 60
+ 5 reserved superuser = 65
→ max_connections = 70

PgBouncer vs pgpool-II

Feature	PgBouncer	pgpool-II
Primary purpose	Connection pooling	Pooling + HA + load balancing
Architecture	Single-process, async (libevent)	Multi-process
Pooling modes	Session / Transaction / Statement	Session / Transaction / Statement
Read/Write splitting	No	Yes (query-based routing)
Replication awareness	No	Yes (streaming + logical)
Connection overhead	Extremely low	Higher
Failover	No (use Patroni/etc.)	Yes (built-in watchdog)
In-database caching	No	Query cache (deprecated in v4.1)
Configuration complexity	Low	High
Community adoption (2024)	Very high	Moderate
Best for	Pure pooling, high throughput	All-in-one HA + pooling

Recommendation: Use PgBouncer for pooling and pair it with Patroni or repmgr for HA. Use pgpool-II only if you specifically need its query-routing or legacy features.

PgBouncer Configuration

Installation

```
# Ubuntu/Debian
sudo apt install pgbouncer

# RHEL/CentOS
sudo yum install pgbouncer
```

```
# macOS (Homebrew)
brew install pgbouncer
```

/etc/pgbouncer/pgbouncer.ini — Full Production Example

```
;; =====
;; pgbouncer.ini - production configuration
;; =====

[databases]
;; Format: alias = host=... port=... dbname=... user=... password=...
;; Leave password blank if using auth_file
myapp_db = host=127.0.0.1 port=5432 dbname=myapp_production

;; Wildcard: any DB name maps to same server DB
;; * = host=127.0.0.1 port=5432

;; Read replica pool (use different alias)
myapp_db_ro = host=replica.internal port=5432 dbname=myapp_production

[pgbouncer]
;; — Networking —————
listen_addr = 127.0.0.1          ;; bind address (use 0.0.0.0 with firewall)
listen_port = 6432              ;; PgBouncer default port
unix_socket_dir = /var/run/postgresql

;; — Pooling Mode —————
pool_mode = transaction         ;; session | transaction | statement

;; — Pool Sizes —————
max_client_conn = 10000         ;; max simultaneous client connections
default_pool_size = 20          ;; server connections per (db, user) pair
min_pool_size = 5               ;; keep this many idle server connections warm
reserve_pool_size = 5           ;; extra connections for emergencies
reserve_pool_timeout = 5.0      ;; seconds to wait before using reserve pool

;; — Timeouts —————
server_connect_timeout = 15     ;; seconds to wait for new server connection
server_idle_timeout = 600       ;; close idle server connections after 10 min
client_idle_timeout = 0         ;; 0 = disabled; set to 60 for strict cleanup
query_timeout = 0               ;; 0 = disabled; set limit (e.g. 30) to kill long
queries
query_wait_timeout = 120        ;; client waits max 120s for a free server conn
idle_transaction_timeout = 0    ;; 0 = disabled; kills idle-in-transaction sessions

;; — Authentication —————
auth_type = scram-sha-256       ;; md5 | scram-sha-256 | trust | hba
auth_file = /etc/pgbouncer/userlist.txt
;; auth_hba_file = /etc/pgbouncer/pg_hba.conf ;; if auth_type = hba

;; — TLS —————
```

```

;; client_tls_sslmode = require
;; client_tls_key_file = /etc/ssl/pgbouncer.key
;; client_tls_cert_file = /etc/ssl/pgbouncer.crt
;; server_tls_sslmode = require

;; — Admin —————
admin_users = pgbouncer_admin
stats_users = monitoring_user    ;; users allowed to run SHOW commands
logfile = /var/log/pgbouncer/pgbouncer.log
pidfile = /var/run/pgbouncer/pgbouncer.pid

;; — Performance —————
server_reset_query = DISCARD ALL ;; run on server conn return (session mode)
;; For transaction mode, reset is lightweight:
;; server_reset_query =          ;; empty for transaction mode (faster)
server_check_query = select 1    ;; health check query
server_check_delay = 30          ;; check every 30s

;; — Logging —————
log_connections = 0              ;; 1 logs every connection (noisy)
log_disconnections = 0
log_pooler_errors = 1
stats_period = 60                ;; emit stats every 60s

```

/etc/pgbouncer/userlist.txt

```

;; Format: "username" "hashed_password_or_plaintext"
;; Generate hash: SELECT concat('md5', md5('password' || 'username'));
;; Or for scram: use pgbouncer's own SCRAM support (pgbouncer >= 1.14)

"app_user" "SCRAM-SHA-256$4096:..."
"readonly_user" "md5abc123..."
"pgbouncer_admin" "md5def456..."

```

Systemd Service

```

# Enable and start
sudo systemctl enable pgbouncer
sudo systemctl start pgbouncer

# Reload config without dropping connections
sudo systemctl reload pgbouncer
# or: kill -HUP $(cat /var/run/pgbouncer/pgbouncer.pid)

```

Monitoring PgBouncer

Connect to the admin console:

```
psql -h 127.0.0.1 -p 6432 -U pgbouncer_admin pgbouncer
```

SHOW POOLS

```
SHOW POOLS;
```

```
database | user | cl_active | cl_waiting | sv_active | sv_idle | sv_used |
sv_tested | sv_login | maxwait | pool_mode
-----+-----+-----+-----+-----+-----+-----+-----
myapp_db | app_user | 45 | 2 | 20 | 0 | 0 |
0 | 0 | 3 | transaction
```

Column	Meaning
cl_active	Clients linked to a server connection and executing
cl_waiting	Clients waiting for a free server connection
sv_active	Server connections in use by a client
sv_idle	Server connections idle, ready for use
sv_used	Server connections idle > server_check_delay
maxwait	Seconds the oldest waiting client has been waiting — key alert metric

Alert rule: If `cl_waiting > 0` AND `maxwait > 5`, you need more server connections.

SHOW STATS

```
SHOW STATS;
```

```
database | total_xact_count | total_query_count | total_received | total_sent |
total_xact_time | avg_xact_time | avg_query_time
-----+-----+-----+-----+-----+-----+-----
myapp_db | 1234567 | 3456789 | 987654321 | 123456789 |
9876543210 | 8012 | 2856
```

Useful derived metrics:

```
avg_query_time (µs) = total_xact_time / total_query_count
queries_per_second = delta(total_query_count) / interval
```

SHOW CLIENTS

```
SHOW CLIENTS;
```


Shows individual client connections with their state, query, and wait time.

SHOW SERVERS

```
SHOW SERVERS;
```

Shows individual server connections — useful to see if connections are stuck `active` .

SHOW CONFIG

```
SHOW CONFIG;
```

Live view of all configuration parameters (read-only, no restart needed for some changes).

Runtime Reconfiguration

```
-- Change pool size without restart
SET default_pool_size = 30;

-- Reconnect all server connections (e.g., after PostgreSQL config change)
RECONNECT myapp_db;

-- Kill a single client
KILL client_addr;

-- Pause all pools (for maintenance)
PAUSE;
RESUME;
```

max_connections in postgresql.conf

Memory Calculation

```
-- Check current setting
SHOW max_connections;

-- Estimate memory reserved
SELECT
    max_connections,
    shared_buffers,
    round(max_connections * 10) AS approx_conn_overhead_mb,
    round(current_setting('shared_buffers')::text::bigint / 1024 / 1024) AS
shared_buffers_mb
FROM pg_settings
WHERE name IN ('max_connections', 'shared_buffers')
GROUP BY max_connections, shared_buffers;
```

Safe max_connections Values

RAM	Recommended max_connections
4 GB	100
8 GB	200
16 GB	300-400
32 GB	500-600
64 GB	700-900

Rule of thumb: $\text{max_connections} \times 10 \text{ MB} \leq 20\% \text{ of total RAM}$

postgresql.conf Tuning

```
# postgresql.conf

max_connections = 200          # absolute ceiling - backends + superusers
superuser_reserved_connections = 3 # reserved for pg_dump, maintenance

# With PgBouncer in front, set this LOW:
# max_connections = 100        # PgBouncer pools never exceed this
# superuser_reserved_connections = 5
```

Common Mistakes

1. **Setting max_connections = 1000 without PgBouncer** — each connection uses ~10 MB, destroying performance under load.
2. **Using session pooling for a stateless API** — you get almost none of the multiplexing benefit.
3. **Using transaction pooling with session-level SET** — settings silently bleed across clients because the server connection is reused.
4. **Not setting idle_transaction_timeout** — a transaction left open holds a server connection indefinitely, exhausting the pool.
5. **Setting pool_size too high** — more server connections than $\text{num_cores} \times 2$ increases lock contention and context switching.
6. **Forgetting server_reset_query** — in session mode, state from one client (e.g., `SET search_path`) can leak to the next client on pool reconnection.
7. **Not monitoring maxwait** — client starvation is invisible without this metric.
8. **Running PgBouncer on the same host as PostgreSQL in production** — single point of failure; run PgBouncer on application servers or a dedicated proxy tier.

Best Practices

- Use **transaction pooling** for stateless web APIs and microservices

- Use **session pooling** for background jobs, analytics, or anything using `LISTEN / NOTIFY`
- Set `idle_transaction_timeout = 30` to kill runaway transactions
- Set `query_wait_timeout = 30` so clients fail fast rather than queue forever
- Monitor `maxwait` and alert at > 1 second
- Keep PgBouncer's `max_client_conn` high (10000+) — it handles client connections cheaply
- Use `server_reset_query = (empty)` in transaction mode for performance — skip `DISCARD ALL`
- Deploy PgBouncer on the **application side** (sidecar or same host as app), not on the DB server
- Use `auth_type = scram-sha-256` — avoid `trust` and `md5` in production
- Version-pin your `pgbouncer.ini` in version control

Performance Considerations

Benchmark: Direct vs PgBouncer Transaction Mode

Scenario: 500 concurrent clients, simple SELECT, 8-core server

Mode	Throughput (TPS)	Latency p99 (ms)	Postgres CPU %
Direct (500 conns)	8,200	245	78%
PgBouncer session	21,400	89	42%
PgBouncer transaction	48,300	31	38%

Connection Overhead vs Query Time

If your average query takes 1 ms and connection setup takes 20 ms, you're spending **95% of time on overhead**. PgBouncer eliminates this.

TLS Termination

PgBouncer can terminate TLS from clients and use unencrypted connections to PostgreSQL on the same host, reducing encryption overhead from connection-heavy workloads.

Interview Questions & Answers

Q1: Why does PostgreSQL use a process-per-connection model instead of threads?

A: PostgreSQL was designed for strong isolation and crash safety. If a backend segfaults, it cannot corrupt the memory of other backends — they are separate OS processes. The shared memory segment (`shared_buffers`, WAL buffers, lock table) is explicitly mapped and protected. Thread-per-connection models have higher throughput per connection but risk heap corruption from buggy query execution code.

Q2: What is the difference between transaction and session pooling?

A: In session pooling, a server connection is held for the duration of the client's entire session. In transaction pooling, the server connection is returned to the pool as soon as `COMMIT / ROLLBACK` is issued. Transaction pooling allows far more clients to share fewer server connections, but disallows session-level features like `SET`, `LISTEN`, session advisory locks, and named prepared statements.

Q3: Why is `pool_size = num_cores × 2` + spindles the recommended formula?

A: PostgreSQL processes compete for CPU and I/O. With `num_cores` connections, all CPU cores are fully utilized for CPU-bound work. The `× 2` factor accounts for I/O wait: while one process waits for disk, another

can use the CPU. `+ spindles` adds connections equal to the number of I/O channels. Adding more connections beyond this only increases lock contention and context-switch overhead without improving throughput.

Q4: What happens when `cl_waiting` is high in SHOW POOLS?

A: Clients are queuing for a server connection. This means the pool is saturated — all `default_pool_size` server connections are in use. Remediation options: increase `default_pool_size` (if PostgreSQL can handle more), increase `max_connections` in PostgreSQL, add read replicas, optimize queries to reduce connection hold time, or check for idle-in-transaction sessions.

Q5: Why should you NOT use session-level SET in transaction pooling mode?

A: In transaction pooling, the server connection is returned to the pool after each transaction and may be reused by a different client. A session-level `SET` persists on the server connection, so the next client that reuses that connection will inherit the modified GUC. Use `SET LOCAL` (which is transaction-scoped and rolled back on COMMIT/ROLLBACK) instead.

Q6: How do you handle LISTEN/NOTIFY with PgBouncer?

A: LISTEN/NOTIFY requires a dedicated, persistent server connection — it cannot work through PgBouncer in transaction pooling mode. Solutions: (a) bypass PgBouncer and connect directly to PostgreSQL for the listener connection, (b) use PgBouncer in session pooling mode for the listener, (c) use `SKIP_LOCKED` polling pattern instead of LISTEN, or (d) use an external message broker.

Q7: What is `reserve_pool_size` and when is it used?

A: It is an extra bank of server connections beyond `default_pool_size`. PgBouncer opens connections from the reserve pool when all default pool connections are in use AND the client wait time exceeds `reserve_pool_timeout` (default 5 seconds). It acts as a burst buffer for traffic spikes without permanently holding extra server connections.

Q8: How does `server_reset_query` differ between session and transaction mode?

A: In session mode, `DISCARD ALL` is run on the server connection when a client disconnects, resetting all session state (temporary tables, prepared statements, advisory locks, `SET` values). In transaction mode, this is not needed because transaction-scoped state is already cleaned up on COMMIT/ROLLBACK. Setting `server_reset_query = (empty)` in transaction mode eliminates a round-trip to PostgreSQL on every connection return, improving throughput.

Q9: What is the `max_connections` formula when PgBouncer is in use?

A: $\text{max_connections} = (\text{number_of_pgbouncer_instances} \times \text{default_pool_size} \times \text{number_of_pools}) + \text{superuser_reserved_connections}$. Since PgBouncer multiplexes many clients onto few server connections, `max_connections` in PostgreSQL can be set much lower — typically 100–200 even with thousands of application clients.

Q10: How do you do a zero-downtime PgBouncer config reload?

A: Send SIGHUP to the PgBouncer process (`kill -HUP <pid>` or `systemctl reload pgbouncer`). Most configuration parameters are applied immediately without dropping existing connections. Changes to `listen_addr`, `listen_port`, or `unix_socket_dir` require a full restart. The `RELOAD SQL` command in the admin console also works.

Exercises with Solutions

Exercise 1: Size a Pool

Problem: Your server has 16 CPU cores and an NVMe SSD. You have 3 separate application databases. What should `default_pool_size` be, and what should `max_connections` be in `postgresql.conf` ?

Solution:

```
pool_size = 16 × 2 + 1 = 33 → round to 35
```

```
For 3 databases × 35 connections each = 105 server connections  
+ 5 superuser reserved = 110
```

```
max_connections = 110 (in postgresql.conf)  
default_pool_size = 35 (in pgbouncer.ini)
```

Exercise 2: Diagnose a Saturated Pool

Problem: Your monitoring shows `maxwait = 8` seconds. What do you check and what actions do you take?

Solution:

```
-- Step 1: Check pool state  
SHOW POOLS;  
-- Look at cl_waiting, sv_active, sv_idle  
  
-- Step 2: Check for idle-in-transaction sessions  
SHOW SERVERS;  
-- Look for sv_active connections that have been active a long time  
  
-- Step 3: On PostgreSQL side, check for locks  
SELECT pid, wait_event_type, wait_event, state, query_start, query  
FROM pg_stat_activity  
WHERE state = 'idle in transaction'  
ORDER BY query_start;  
  
-- Step 4: Immediate action – set idle_transaction_timeout  
-- In pgbouncer.ini:  
-- idle_transaction_timeout = 30  
  
-- Step 5: Long-term – increase pool size if queries are short  
-- or optimize queries to reduce connection hold time
```

Exercise 3: Write pgbouncer.ini for Microservice

Problem: Write a minimal `pgbouncer.ini` for a stateless REST API with 50 concurrent workers, connecting to PostgreSQL on `db.internal:5432`, database `orders`.

Solution:

[databases]

orders = host=db.internal port=5432 dbname=orders

[pgbouncer]

```
listen_addr = 127.0.0.1
listen_port = 6432
pool_mode = transaction
max_client_conn = 500
default_pool_size = 20
min_pool_size = 5
reserve_pool_size = 5
reserve_pool_timeout = 3
server_idle_timeout = 300
idle_transaction_timeout = 30
query_wait_timeout = 30
auth_type = scram-sha-256
auth_file = /etc/pgbouncer/userlist.txt
server_reset_query =
logfile = /var/log/pgbouncer/pgbouncer.log
```

Cross-References

- [02_orm_optimization.md](#) — How ORMs interact with connection pools (connection acquisition in ORM layers)
- [03_transactions_in_apps.md](#) — Transaction management and SAVEPOINT support in pooling modes
- [09_java_spring_postgresql.md](#) — HikariCP (application-level pool) + PgBouncer (server-side pool) layering
- [10_nodejs_postgresql.md](#) — `pg.Pool` configuration and interaction with PgBouncer
- [11_python_postgresql.md](#) — `psycopg2`/`psycopg3` connection pooling
- [08_scaling_patterns.md](#) — How pooling interacts with read replica routing